

KẾ HOẠCH THIẾT KẾ HỆ THỐNG MẠNG XÃ HỘI NHẮN TIN

Social Messaging Platform — Technical Design Document

Contents

1. Tổng quan dự án & Mục tiêu.....	3
1.1 Mô tả dự án.....	3
1.2 Phạm vi giai đoạn 1 (MVP)	3
1.3 Mục tiêu phi chức năng	3
2. Kiến trúc hệ thống (System Architecture).....	3
2.1 Tổng quan kiến trúc.....	3
2.2 Các thành phần chính	3
2.3 Mô hình giao tiếp giữa các service.....	4
2.4 Sơ đồ luồng dữ liệu (Data Flow).....	4
3. Tech Stack chi tiết.....	4
3.1 Frontend.....	4
3.2 Backend	4
3.3 Database & Storage.....	5
3.4 Redis — In-Memory Layer (Cache & Real-time).....	5
5 vai trò của Redis trong hệ thống.....	5
Redis Persistence — Không lo mất dữ liệu	6
Nguyên tắc quan trọng: Redis là cache, MySQL là source of truth	6
3.5 Infrastructure & DevOps	6
4. Thiết kế Database chi tiết	7
4.1 User Database (MySQL)	7
Bảng: users	7
Bảng: user_connections (Kết nối nội bộ)	7
Bảng: user_sessions.....	8
4.2 Post Database (MySQL).....	8
Bảng: posts	8
Bảng: post_interactions (Like, Comment, Share)	9
4.3 Messaging Database (MongoDB / ScyllaDB)	9
Collection: conversations	9
Collection: messages	10
4.4 Notification Database (MySQL)	10
Bảng: notifications	10
4.5 Chiến lược Index & Tối ưu Query	11
4.6 Tóm tắt quan hệ giữa các bảng	11
5. Hệ thống xác thực & Bảo mật	12
5.1 Luồng xác thực (Authentication Flow).....	12
Đăng ký (Registration)	12

Đăng nhập (Login)	12
Token Refresh.....	12
5.2 Cấu trúc JWT Token	12
5.3 OAuth 2.0 — Social Login	13
5.4 Các biện pháp bảo mật	13
Bảo mật truyền tải (Transport Security)	13
Bảo mật dữ liệu (Data Security).....	13
Bảo mật API (API Security)	13
Chống tấn công (Attack Prevention)	13
5.5 Audit Logging & Compliance.....	14
6. Thiết kế tính năng chính (Core Features)	14
6.1 Hệ thống đăng bài (Post System)	14
Quy trình tạo bài viết.....	14
Newsfeed Algorithm	14
6.2 Hệ thống nhắn tin (Messaging System).....	15
Kiến trúc real-time messaging	15
Luồng gửi tin nhắn	15
End-to-End Encryption (E2E).....	15
Luồng tạo nhóm chat (Group Creation)	16
Quản lý nhóm chat (Group Management)	17
Phân quyền thành viên nhóm.....	18
Luồng nhắn tin trong nhóm (Group Messaging Flow)	Error! Bookmark not defined.
Giới hạn & Ràng buộc nhóm chat.....	Error! Bookmark not defined.
6.3 Hệ thống kết nối nội bộ (Internal Connection System)	18
Luồng gửi kết bạn	18
Tìm kiếm & Gợi ý kết nối	18
6.4 Hệ thống thông báo (Notification System)	18
7. Quy trình phát triển & CI/CD	19
7.1 Git Branching Strategy	19
7.2 CI Pipeline (GitHub Actions)	19
7.3 CD Pipeline (ArgoCD — GitOps)	19
7.4 Testing Strategy	20
7.5 Monitoring & Observability	20
8. Kế hoạch mở rộng & Tính năng tương lai.....	20
8.1 Kiến trúc mở rộng (Scalability)	20
8.2 Các module tính năng tương lai	21
8.3 Chuẩn bị cho tích hợp thanh toán.....	21
8.4 Roadmap phát triển	22
8.5 Đề xuất cơ cấu đội ngũ (Phase 1).....	22

1. Tổng quan dự án & Mục tiêu

1.1 Mô tả dự án

Xây dựng một nền tảng trong tương lai.

1.2 Phạm vi giai đoạn 1 (MVP)

Các tính năng cốt lõi trong giai đoạn đầu — triển khai cho cả hai tier (Personal & Enterprise):

- Đăng ký / Đăng nhập / Xác thực đa yếu tố (MFA)
- Đăng bài viết (text, hình ảnh, video ngắn) — Newsfeed
- Nhắn tin 1-1 và nhóm (real-time messaging)
- Kết nối nội bộ (friend/follow system, danh bạ tổ chức)
- Thông báo đẩy (Push Notifications)
- Quản lý hồ sơ cá nhân (Profile)

1.3 Mục tiêu phi chức năng

- Bảo mật: Mã hóa end-to-end cho tin nhắn, OAuth 2.0 + JWT, tuân thủ OWASP Top 10
- Hiệu năng: Độ trễ nhắn tin < 200ms, API response < 500ms (p95)
- Khả dụng: Uptime $\geq 99.9\%$, failover tự động
- Mở rộng: Kiến trúc cho phép thêm module mới mà không ảnh hưởng hệ thống hiện tại

2. Kiến trúc hệ thống (System Architecture)

2.1 Tổng quan kiến trúc

Hệ thống áp dụng kiến trúc Microservices kết hợp Event-Driven Architecture. Mỗi domain nghiệp vụ được tách thành một service độc lập, giao tiếp qua API Gateway và Message Broker. Điều này cho phép từng module được phát triển, triển khai và mở rộng độc lập.

2.2 Các thành phần chính

Thành phần	Mô tả	Công nghệ
API Gateway	Điểm vào duy nhất, routing, rate limiting, authentication	Kong / Nginx / AWS API Gateway
Auth Service	Đăng ký, đăng nhập, JWT, MFA, OAuth 2.0	Node.js + Passport.js
User Service	Quản lý hồ sơ, kết nối, danh bạ nội bộ	Node.js / Go
Post Service	CRUD bài viết, newsfeed, media upload	Node.js + S3
Messaging Service	Nhắn tin real-time, nhóm chat, lịch sử	Node.js + Socket.IO / Go
Notification Service	Push notifications, email, in-app alerts	Node.js + Firebase FCM

Media Service	Upload, resize, CDN distribution	Node.js + Sharp + CloudFront
Search Service	Full-text search bài viết, tin nhắn, người dùng	Elasticsearch
Message Broker	Event bus liên service	RabbitMQ / Apache Kafka

2.3 Mô hình giao tiếp giữa các service

- Đồng bộ (Synchronous): REST API qua HTTP/2 cho các thao tác CRUD cơ bản (đăng ký, xem profile, tạo bài viết)
- Bất đồng bộ (Asynchronous): Message Queue (RabbitMQ/Kafka) cho các event như gửi notification, xử lý media, cập nhật newsfeed
- Real-time: WebSocket (Socket.IO) cho nhắn tin và thông báo trực tiếp

2.4 Sơ đồ luồng dữ liệu (Data Flow)

Client (Web/Mobile) → CDN (static assets) → Load Balancer → API Gateway → [Auth Middleware] → Microservice → Database/Cache → Response. Với nhắn tin: Client ↔ WebSocket Server ↔ Message Broker ↔ Messaging Service ↔ Database.

3. Tech Stack chi tiết

3.1 Frontend

Nền tảng	Công nghệ	Lý do chọn
Web App	Next.js 14 (React) + TypeScript	SSR/SSG, SEO tốt, App Router, Server Components
Mobile App	React Native (Expo) + TypeScript	Cross-platform iOS/Android, shared code với Web
State Management	Zustand + React Query (TanStack)	Lightweight, caching tối ưu cho API calls
UI Library	Tailwind CSS + Basuicn/ui (Web) / NativeWind (Mobile)	Consistent design system, utility-first
Real-time Client	Socket.IO Client	Auto-reconnect, fallback transport, room management
Form & Validation	React Hook Form + Zod	Type-safe validation, performance tốt

3.2 Backend

Thành phần	Công nghệ	Lý do chọn
API Framework	NestJS (Node.js) + TypeScript	Modular, DI built-in, microservices support native
Real-time Server	Socket.IO + Redis Adapter	Scalable WebSocket với multi-instance support
ORM / Query Builder	Prisma ORM	Type-safe queries, migration management, multi-DB support

API Documentation	Swagger (OpenAPI 3.0)	Auto-generated từ NestJS decorators
Task Queue	BullMQ (Redis-based)	Job scheduling, retry logic, rate limiting
Logging	Winston + ELK Stack	Structured logging, centralized search

3.3 Database & Storage

Mục đích	Công nghệ	Lý do chọn
Primary Database	MySQL 8.0	ACID compliance, JSON support, wide hosting support, mature ecosystem
Cache Layer	Redis 7 (Cluster mode)	Session storage, message queue, real-time pub/sub, rate limiting
Message Storage	MongoDB (hoặc ScyllaDB)	Schema-flexible cho conversation data, horizontal scaling tốt
Search Engine	Elasticsearch 8	Full-text search bài viết, người dùng — fuzzy matching, ranking
File Storage	AWS S3 / MinIO (self-hosted)	Object storage cho media (ảnh, video, file đính kèm)
CDN	CloudFront / Cloudflare	Global distribution cho static assets và media

3.4 Redis — In-Memory Layer (Cache & Real-time)

Redis là lớp In-Memory trung gian giữa client và database. Redis không thay thế MySQL hay MongoDB — hai hệ thống phục vụ mục đích hoàn toàn khác nhau: MySQL lưu dữ liệu bền vững xuống ổ đĩa (persistent), còn Redis lưu trữ trực tiếp trên RAM để đạt tốc độ đọc/ghi dưới 1 mili-giây. Bỏ Redis khỏi hệ thống này sẽ khiến WebSocket multi-instance không hoạt động và MySQL bị quá tải ngay từ vài trăm user đồng thời.

5 vai trò của Redis trong hệ thống

Vai trò	Cơ chế Redis	Thay thế bằng MySQL?	Lý do không thay được
Session Cache	GET/SET với TTL	Không nên	MySQL ~10–50ms/query; Redis < 1ms. 1000 req/s → MySQL quá tải
Newsfeed Cache	Sorted Set (ZADD/ZRANGEBYSCORE)	Không	MySQL không có cấu trúc sorted set native; query feed mỗi lần = full table scan

Rate Limiting	INCR + EXPIRE (atomic)	Không	Cần atomic increment tốc độ cao; MySQL transaction quá nặng cho workload này
WebSocket Pub/Sub	PUBLISH/SUBSCRIBE channels	Không thể	MySQL không có pub/sub; không thể broadcast event giữa nhiều Socket.IO instances
Presence System	SET với TTL tự expire	Không	Cần TTL tự động khi user mất kết nối; MySQL phải dùng cron job — độ trễ cao

Redis Persistence — Không lo mất dữ liệu

Redis lưu trữ trên RAM nhưng có hai cơ chế ghi xuống ổ đĩa để bảo vệ dữ liệu khi server khởi động lại:

- RDB (Redis Database Snapshot): Dump toàn bộ dataset xuống file .rdb định kỳ (mỗi 60 giây hoặc sau N thay đổi). Phù hợp cho newsfeed cache — mất tối đa 60 giây dữ liệu cache nếu crash
- AOF (Append Only File): Ghi log mọi thao tác write vào file. Khi restart, Redis replay lại log để khôi phục. Recovery gần như đầy đủ (mất < 1 giây)
- Khuyến nghị cho hệ thống này: Bật AOF cho session data (không muốn user bị logout khi Redis restart), RDB snapshot cho newsfeed cache (mất cache thì rebuild từ MySQL — chấp nhận được)

Nguyên tắc quan trọng: Redis là cache, MySQL là source of truth

- Dữ liệu thật (users, posts, messages, notifications) luôn được lưu trước vào MySQL/MongoDB — Redis chỉ là bản sao tăng tốc
- Cache invalidation: Khi dữ liệu MySQL thay đổi (user update profile, post bị xóa) → xóa hoặc cập nhật cache Redis tương ứng
- Cache miss strategy: Nếu Redis không có dữ liệu (cold start hoặc sau restart) → query MySQL → lưu kết quả vào Redis cho lần sau
- Không bao giờ ghi dữ liệu quan trọng chỉ vào Redis mà bỏ qua MySQL (trừ ephemeral data như typing indicator, presence)

3.5 Infrastructure & DevOps

Thành phần	Công nghệ	Ghi chú
Container	Docker + Docker Compose (dev)	Đóng gói từng microservice
Orchestration	Kubernetes (EKS / GKE)	Auto-scaling, self-healing, rolling updates
CI/CD	GitHub Actions + ArgoCD	Automated testing, build, deploy GitOps-style

IaC	Terraform + Helm Charts	Infrastructure as Code, reproducible environments
Monitoring	Prometheus + Grafana	Metrics collection, alerting, dashboards
APM	Jaeger (distributed tracing)	Request tracing across microservices
Secret Management	HashiCorp Vault / AWS Secrets Manager	Centralized secret rotation

4. Thiết kế Database chi tiết

Database được thiết kế theo nguyên tắc Database-per-Service: mỗi microservice sở hữu database riêng, không truy cập trực tiếp vào database của service khác. Giao tiếp dữ liệu qua API hoặc event. Hệ thống sử dụng MySQL 8.0 (thay vì PostgreSQL) — tương thích đầy đủ với Prisma ORM, lưu ý: UUID được generate ở application layer, kiểu JSONB thay bằng JSON, kiểu INET thay bằng VARCHAR(45).

4.1 User Database (MySQL)

Bảng: users

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK (UUID via app)	ID người dùng
email	VARCHAR(255)	UNIQUE, NOT NULL	Email đăng nhập
phone	VARCHAR(20)	UNIQUE, NULLABLE	Số điện thoại
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hash (cost=12)
display_name	VARCHAR(100)	NOT NULL	Tên hiển thị
username	VARCHAR(50)	UNIQUE, NOT NULL	Username duy nhất
avatar_url	TEXT	NULLABLE	URL ảnh đại diện (S3)
bio	VARCHAR(500)	NULLABLE	Tiểu sử ngắn
status	ENUM	DEFAULT 'active'	active / suspended / deactivated
role	ENUM	DEFAULT 'member'	admin / moderator / member
email_verified	BOOLEAN	DEFAULT false	Đã xác thực email chưa
mfa_enabled	BOOLEAN	DEFAULT false	Bật xác thực 2 yếu tố
mfa_secret	VARCHAR(255)	NULLABLE, ENCRYPTED	TOTP secret (mã hóa AES-256)
last_seen_at	TIMESTAMP	NULLABLE	Lần online cuối
created_at	TIMESTAMP	DEFAULT NOW()	Ngày tạo tài khoản
updated_at	TIMESTAMP	AUTO UPDATE	Ngày cập nhật cuối

Bảng: user_connections (Kết nối nội bộ)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID kết nối

requester_id	UUID	FK → users.id	Người gửi yêu cầu
addressee_id	UUID	FK → users.id	Người nhận yêu cầu
status	ENUM	DEFAULT 'pending'	pending / accepted / blocked
type	ENUM	DEFAULT 'friend'	friend / follow / colleague
created_at	TIMESTAMP	DEFAULT NOW()	Ngày gửi yêu cầu
updated_at	TIMESTAMP	AUTO UPDATE	Ngày cập nhật

Bảng: user_sessions

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	Session ID
user_id	UUID	FK → users.id, INDEX	Người dùng
refresh_token_hash	VARCHAR(255)	NOT NULL	Hash của refresh token
device_info	JSON	NULLABLE	User-Agent, device type, OS
ip_address	VARCHAR(45)	NOT NULL	IP đăng nhập
expires_at	TIMESTAMP	NOT NULL	Thời hạn session
created_at	TIMESTAMP	DEFAULT NOW()	Ngày tạo

4.2 Post Database (MySQL)

Bảng: posts

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID bài viết
author_id	UUID	FK → users.id, INDEX	Người đăng
content	TEXT	NOT NULL, MAX 5000 chars	Nội dung bài viết
media_urls	JSON	NULLABLE	Array URLs media [{url, type, thumbnail}]
visibility	ENUM	DEFAULT 'connections'	public / connections / private / group
group_id	UUID	FK → groups.id, NULLABLE	Nhóm (nếu đăng trong group)
like_count	INTEGER	DEFAULT 0	Counter cache — số lượt thích
comment_count	INTEGER	DEFAULT 0	Counter cache — số bình luận
share_count	INTEGER	DEFAULT 0	Counter cache — số lượt chia sẻ
is_pinned	BOOLEAN	DEFAULT false	Ghim bài viết
is_deleted	BOOLEAN	DEFAULT false	Soft delete

created_at	TIMESTAMP	DEFAULT NOW(), INDEX	Ngày đăng (dùng cho feed sorting)
updated_at	TIMESTAMP	AUTO UPDATE	Ngày chỉnh sửa cuối

Bảng: post_interactions (Like, Comment, Share)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PK	ID tương tác
post_id	UUID	FK → posts.id, INDEX	Bài viết
user_id	UUID	FK → users.id, INDEX	Người tương tác
type	ENUM	NOT NULL	like / comment / share
content	TEXT	NULLABLE	Nội dung bình luận (nếu type=comment)
parent_id	UUID	FK → post_interactions.id, NULLABLE	Reply to comment (nested comments)
created_at	TIMESTAMP	DEFAULT NOW()	Ngày tạo

4.3 Messaging Database (MongoDB / ScyllaDB)

Tin nhắn sử dụng NoSQL để tối ưu write-heavy workload và horizontal scaling. Schema được thiết kế cho partition by conversation_id, sắp xếp theo timestamp.

Collection: conversations

Field	Type	Index	Mô tả
_id	ObjectId	PK	ID cuộc hội thoại
type	String		direct / group
name	String		Tên nhóm (group chat)
avatar_url	String		Ảnh nhóm
participants	Array<Object>	Compound Index	{user_id, role, joined_at, last_read_at}
created_by	ObjectId		Người tạo
last_message	Object		{content, sender_id, sent_at} — denormalized cho list view
is_encrypted	Boolean		E2E encrypted conversation disappearing_timer Number Timer tự xóa tin nhắn (đơn vị giây) áp dụng cho toàn bộ

			conversation. 0 hoặc null = tắt. Ví dụ: 86400 = 24 giờ, 604800 = 7 ngày. Tin nhắn mới gửi sẽ được gán auto_delete_at = sent_at + disappearing_timer.
created_at	Date		Ngày tạo
updated_at	Date	INDEX (sort)	Sort conversations by recent activity

Collection: messages

Field	Type	Index	Mô tả
_id	ObjectId	PK	ID tin nhắn
conversation_id	ObjectId	Compound (conv_id + sent_at)	Cuộc hội thoại chứa tin nhắn
sender_id	UUID (ref)		Người gửi
type	String		text / image / video / file / system
content	String		Nội dung (hoặc ciphertext nếu E2E)
media	Object		{url, type, size, thumbnail_url}
reply_to	ObjectId		Tin nhắn đang reply
reactions	Array<Object>		[{user_id, emoji, created_at}]
read_by	Array<Object>		[{user_id, read_at}]
status	String		sent / delivered / read
is_deleted	Boolean		Soft delete (hiển thị 'tin nhắn đã xóa') auto_delete_at Date Thời điểm tự xóa tin nhắn — do disappearing message timer của conversation quy định. Null nếu không áp dụng.
sent_at	Date	INDEX (DESC)	Thời gian gửi — query gần nhất trước
edited_at	Date		Thời gian chỉnh sửa

4.4 Notification Database (MySQL)

Bảng: notifications

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
-----	--------------	-----------	-------

id	UUID	PK	ID thông báo
recipient_id	UUID	FK → users.id, INDEX	Người nhận
actor_id	UUID	FK → users.id	Người thực hiện hành động
type	ENUM	NOT NULL	like / comment / follow_request / message / mention / system
entity_type	VARCHAR(50)	NOT NULL	post / comment / conversation / user
entity_id	UUID	NOT NULL	ID của entity liên quan
content	VARCHAR(500)	NOT NULL	Nội dung thông báo (pre-rendered)
is_read	BOOLEAN	DEFAULT false, INDEX	Đã đọc chưa
read_at	TIMESTAMP	NULLABLE	Thời gian đọc
created_at	TIMESTAMP	DEFAULT NOW(), INDEX	Ngày tạo — sort DESC

4.5 Chiến lược Index & Tối ưu Query

- MySQL Composite Indexes: (author_id, created_at DESC) trên posts — tối ưu query feed của một user
- MySQL Filtered Query (không có Partial Index): Dùng Generated Column hoặc query kèm WHERE is_deleted = 0 — MySQL không hỗ trợ partial index như PostgreSQL
- MongoDB Compound Index: (conversation_id, sent_at DESC) trên messages — phân trang tin nhắn theo cuộc hội thoại
- MongoDB TTL Index: Tự động xóa messages cũ hơn retention period (nếu cấu hình)
- Redis Sorted Sets: Newsfeed cache — ZADD feed:{user_id} score=timestamp member=post_id
- Elasticsearch (bắt buộc với MySQL): Full-text index trên posts.content và users.display_name — MySQL built-in FULLTEXT không hỗ trợ tiếng Việt Unicode tốt, Elasticsearch với ICU analyzer là lựa chọn thay thế bắt buộc
- Database Partitioning: Partition bảng messages theo tháng (range partition on sent_at) khi data > 100M rows

4.6 Tóm tắt quan hệ giữa các bảng

- users 1 ↔ N posts (một user có nhiều bài viết)
- users N ↔ N users qua user_connections (quan hệ kết nối hai chiều)
- users 1 ↔ N user_sessions (nhiều device đăng nhập cùng lúc)
- posts 1 ↔ N post_interactions (like, comment, share)
- users N ↔ N conversations qua participants array
- conversations 1 ↔ N messages (một hội thoại chứa nhiều tin nhắn)
- users 1 ↔ N notifications (một user nhận nhiều thông báo)

5. Hệ thống xác thực & Bảo mật

5.1 Luồng xác thực (Authentication Flow)

Đăng ký (Registration)

- ❖ User gửi email/phone + password → API Gateway → Auth Service
- ❖ Validate input (Zod schema): email format, password strength (min 8 chars, 1 uppercase, 1 number, 1 special)
- ❖ Hash password với bcrypt (cost factor = 12)
- ❖ Tạo record trong bảng users (status = 'pending_verification')
- ❖ Gửi email verification link chứa JWT token (exp: 24h) qua Notification Service
- ❖ User click link → Auth Service verify token → update email_verified = true
- ❖ Redirect về trang đăng nhập

Đăng nhập (Login)

- ❖ User gửi email + password → Auth Service
- ❖ Lookup user by email → so sánh bcrypt hash
- ❖ Nếu MFA enabled → gửi TOTP challenge, user nhập mã từ Authenticator App
- ❖ Generate token pair: Access Token (JWT, exp: 15 phút) + Refresh Token (opaque, exp: 30 ngày)
- ❖ Lưu refresh_token_hash + device_info vào bảng user_sessions
- ❖ Trả về {access_token, refresh_token, user_profile} cho client
- ❖ Client lưu access_token trong memory, refresh_token trong HTTP-only Secure cookie

Token Refresh

- ❖ Client gửi refresh_token (từ cookie) khi access_token hết hạn (401 response)
- ❖ Auth Service: hash refresh_token → lookup trong user_sessions
- ❖ Validate: chưa expired, device_info khớp, chưa bị revoke
- ❖ Token Rotation: Generate access_token mới + refresh_token mới, invalidate token cũ
- ❖ Nếu phát hiện refresh_token đã được sử dụng lại (replay attack) → revoke toàn bộ sessions của user + alert

5.2 Cấu trúc JWT Token

Trường	Giá trị	Mô tả
iss	social-platform-auth	Issuer — service phát hành
sub	user UUID	Subject — ID người dùng
aud	social-platform-api	Audience — API được phép sử dụng
exp	15 phút từ lúc phát hành	Expiration — ngăn để giảm thiểu rủi ro
iat	timestamp hiện tại	Issued At
jti	UUID ngẫu nhiên	JWT ID — chống replay, hỗ trợ blacklist
role	member / admin / moderator	Authorization role
permissions	string[]	Danh sách quyền cụ thể (fine-grained)
device_id	hash(user-agent + fingerprint)	Bind token với device

Signing Algorithm: RS256 (RSA asymmetric). Public key được publish tại /.well-known/jwks.json để các service khác verify mà không cần shared secret.

5.3 OAuth 2.0 — Social Login

Hỗ trợ đăng nhập qua Google, Apple, Microsoft (Azure AD) bằng OAuth 2.0 Authorization Code Flow + PKCE. Không sử dụng Implicit Flow vì lỗ hổng bảo mật đã biết.

- ✓ Client redirect user → Provider Authorization URL (với code_challenge PKCE)
- ✓ User đồng ý → Provider redirect về callback URL kèm authorization_code
- ✓ Auth Service gửi code + code_verifier → Provider Token Endpoint → nhận ID token
- ✓ Verify ID token (signature + claims) → Tìm hoặc tạo user account (link with existing if email match)
- ✓ Generate JWT pair và login bình thường

5.4 Các biện pháp bảo mật

Bảo mật truyền tải (Transport Security)

- TLS 1.3 bắt buộc cho tất cả API endpoints và WebSocket connections
- HSTS header (max-age=31536000, includeSubDomains, preload)
- Certificate Pinning trên mobile app (backup pin cho certificate rotation)

Bảo mật dữ liệu (Data Security)

- Encryption at rest: AES-256 cho database (MySQL InnoDB Tablespace Encryption / MongoDB encrypted storage engine)
- Encryption in transit: TLS 1.3 cho tất cả internal service communication
- End-to-End Encryption (E2E) cho nhắn tin: Signal Protocol (Double Ratchet) — server không thể đọc nội dung
- PII Masking: Che dấu email/phone trong API response (v***@gmail.com)
- Field-level encryption: mfa_secret, recovery_codes được mã hóa riêng bằng application-level AES-256-GCM
- Key Management: Rotation key mỗi 90 ngày qua HashiCorp Vault

Bảo mật API (API Security)

- Rate Limiting: 100 requests/phút per user (Redis sliding window), 10 requests/phút cho login endpoint
- CORS: Whitelist chỉ domain chính thức, không wildcard (*)
- CSRF Protection: Double-submit cookie pattern cho Web, SameSite=Strict
- Input Validation: Zod schema validation ở cả client và server — reject malformed requests trước khi tới business logic
- SQL Injection Prevention: Parameterized queries (Prisma ORM) — không raw string concatenation
- XSS Prevention: Content Security Policy (CSP), HTML sanitization (DOMPurify) cho user-generated content
- File Upload Security: Whitelist MIME types, virus scanning (ClamAV), max size 50MB, store với random UUID filename

Chống tấn công (Attack Prevention)

Loại tấn công	Biện pháp phòng chống
---------------	-----------------------

Brute Force Login	Account lockout sau 5 lần sai (30 phút), CAPTCHA sau 3 lần, exponential backoff
Credential Stuffing	Breached password check (Have I Been Pwned API), mandatory MFA cho admin
Session Hijacking	Token binding (device_id), IP-based anomaly detection, refresh token rotation
DDoS	CloudFlare/AWS Shield, WAF rules, rate limiting multi-layer
Man-in-the-Middle	TLS 1.3, HSTS, Certificate Pinning (mobile)
Insecure Direct Object Reference	UUID thay vì sequential ID, authorization check mỗi request
Data Exfiltration	Audit logging mọi data access, anomaly detection cho bulk queries

5.5 Audit Logging & Compliance

- Ghi log tất cả authentication events (login, logout, failed attempts, MFA changes) — immutable append-only log
- Log format: {timestamp, user_id, action, ip, device, result, metadata} — structured JSON
- Retention: Auth logs giữ 2 năm, access logs giữ 90 ngày
- GDPR compliance: Right to erasure (xóa PII khi user yêu cầu), data portability (export user data dạng JSON)
- Mọi admin action được log riêng trong admin_audit_logs với lý do thực hiện

6. Thiết kế tính năng chính (Core Features)

6.1 Hệ thống đăng bài (Post System)

Quy trình tạo bài viết

- ✓ User soạn bài → Client validate (content length, media count ≤ 10)
- ✓ Upload media song song lên Media Service → nhận presigned S3 URLs
- ✓ Media Service: resize ảnh (thumbnail 200px, medium 800px, original), transcode video (HLS adaptive bitrate)
- ✓ Gửi POST /api/v1/posts {content, media_urls, visibility} → Post Service
- ✓ Post Service: Sanitize HTML content (DOMPurify), extract @mentions và #hashtags
- ✓ Lưu vào MySQL, publish event 'post.created' lên Message Broker
- ✓ Newsfeed Service nhận event → fan-out: push post_id vào Redis sorted set của mỗi follower/connection
- ✓ Notification Service nhận event → tạo notification cho @mentioned users
- ✓ Client nhận push notification hoặc poll newsfeed endpoint

Newsfeed Algorithm

Kết hợp hai phương pháp: Fan-out on Write cho users có < 1000 connections (write post_id vào feed cache của từng follower khi đăng), và Fan-out on Read cho users có > 1000 connections (query lúc load feed).

Ranking score tính theo công thức:

$$score = (like_weight \times likes + comment_weight \times comments + share_weight \times shares) \times time_decay(hours_since_post) \times relationship_score(viewer, author)$$

Cache strategy: Redis Sorted Set (ZRANGEBYSCORE) lưu top 500 posts per user feed. TTL = 1 giờ.

Cache miss → query MySQL với composite index (created_at DESC, author_id).

6.2 Hệ thống nhắn tin (Messaging System)

Kiến trúc real-time messaging

- Kết nối: Client ↔ WebSocket (Socket.IO) ↔ Redis Pub/Sub ↔ Messaging Service ↔ MongoDB
- Multi-instance support: Redis Adapter cho Socket.IO — user kết nối vào bất kỳ server nào đều nhận tin
- Presence system: Redis SET lưu online users, PUBLISH khi user online/offline
- Typing indicator: Ephemeral event qua WebSocket (không lưu DB), debounce 2 giây
- Delivery receipts: sent → delivered (khi target device nhận qua WebSocket ACK) → read (khi user mở conversation)

Luồng gửi tin nhắn

- Client emit 'message:send' qua WebSocket {conversation_id, content, type, reply_to}
- WebSocket Server authenticate (verify JWT từ connection handshake)
- Nếu E2E encrypted: client đã encrypt bằng recipient's public key trước khi gửi — server chỉ forward ciphertext
- Messaging Service lưu message vào MongoDB (status = 'sent')
- Publish event lên Redis channel: 'conversation:{id}'
- Tất cả Socket.IO instances subscribe channel → emit 'message:new' đến online participants
- Nếu recipient offline → push event qua Notification Service → Firebase FCM / APNs
- Client nhận ACK → update UI (tick xanh = delivered, tick đôi = read)

End-to-End Encryption (E2E)

- Media Visibility Options (Chế độ xem media)
- Khi gửi ảnh, video hoặc file, người gửi có thể chọn một trong ba chế độ hiển thị:
-
- Permanent (xem vĩnh viễn — mặc định): Media lưu bình thường trên S3, người nhận xem và tải lại bất kỳ lúc nào.
-
- View Once (xem 1 lần): view_mode = once. Client khóa UI không cho screenshot (overlay bảo vệ trên mobile). Sau khi người nhận mở và đóng, Messaging Service cập nhật is_deleted = true và xóa file khỏi S3 (soft-delete metadata, hard-delete file). Người gửi thấy trạng thái 'Đã xem 1 lần'.
-
- Timed View (xem trong khoảng thời gian tự đặt): view_mode = timed, người gửi đặt timer (ví dụ: 10 giây, 1 phút, 1 giờ). Server ghi expires_at = sent_at + duration. Redis TTL job kiểm tra định kỳ — sau expires_at: ẩn media trên client, xóa file khỏi S3. Người nhận thấy 'Tin nhắn đã hết hạn'.
-
- Ghi chú kỹ thuật: View Once và Timed View chỉ áp dụng cho media (image/video/file), không áp dụng cho text. Server-side enforcement là source of truth — client chỉ render, không tự quyết định trạng thái. Khi E2E bật, file được encrypt trước khi upload S3, view_mode logic vẫn được xử lý bình thường trên metadata.
-
- End-to-End Encryption (E2E)

- Disappearing Messages (Tin nhắn tự xóa)
- Tính năng đặt timer tự xóa toàn bộ tin nhắn trong một cuộc hội thoại — áp dụng cho cả text lẫn media, khác với View Once (chỉ áp dụng cho media từng tin).
-
- Cách hoạt động: Admin hoặc người tạo conversation bật tính năng và chọn thời gian: 10 giây | 1 phút | 1 giờ | 24 giờ | 7 ngày | 30 ngày. Mọi tin nhắn mới sẽ được gán `auto_delete_at = sent_at + timer`. Redis TTL job quét định kỳ (mỗi 30 giây) — khi đến `auto_delete_at`: cập nhật `is_deleted = true`, xóa file S3 nếu là media, push sự kiện `message:deleted` qua WebSocket để client ẩn tin nhắn ngay lập tức.
-
- Timer tính từ lúc gửi hay lúc đọc: Mặc định tính từ `sent_at`. Tùy chọn nâng cao (Phase 2): tính từ lúc đọc — `auto_delete_at` chỉ được gán sau khi người nhận mở tin nhắn lần đầu.
-
- Ghi chú: Tin nhắn cũ trước khi bật timer không bị xóa ngược. Client hiển thị icon đồng hồ và countdown trên mỗi tin nhắn. Screenshot vẫn có thể thực hiện — đây là giới hạn kỹ thuật chung của mọi nền tảng.
-
- Media Visibility Options (Chế độ xem media)
- Khi gửi ảnh, video hoặc file, người gửi có thể chọn một trong ba chế độ hiển thị:
-
- Permanent (xem vĩnh viễn — mặc định): Media lưu bình thường trên S3, người nhận xem và tải lại bất kỳ lúc nào.
-
- View Once (xem 1 lần): `view_mode = once`. Client khóa UI không cho screenshot (overlay bảo vệ trên mobile). Sau khi người nhận mở và đóng, Messaging Service cập nhật `is_deleted = true` và xóa file khỏi S3 (soft-delete metadata, hard-delete file). Người gửi thấy trạng thái 'Đã xem 1 lần'.
-
- Timed View (xem trong khoảng thời gian tự đặt): `view_mode = timed`, người gửi đặt timer (ví dụ: 10 giây, 1 phút, 1 giờ). Server ghi `expires_at = sent_at + duration`. Redis TTL job kiểm tra định kỳ — sau `expires_at`: ẩn media trên client, xóa file khỏi S3. Người nhận thấy 'Tin nhắn đã hết hạn'.
-
- Ghi chú kỹ thuật: View Once và Timed View chỉ áp dụng cho media (image/video/file), không áp dụng cho text. Server-side enforcement là source of truth. Khi E2E bật, file được encrypt trước khi upload S3, `view_mode` logic vẫn được xử lý bình thường trên metadata.
-
- End-to-End Encryption (E2E)
- Protocol: Signal Protocol (Double Ratchet Algorithm) — forward secrecy + break-in recovery
- Key Exchange: X3DH (Extended Triple Diffie-Hellman) khi bắt đầu conversation mới
- Mỗi user có Identity Key (long-term), Signed Pre-Key (medium-term), One-Time Pre-Keys (ephemeral)
- Server lưu public keys tại `/api/v1/keys/{user_id}` — KHÔNG lưu private keys
- Group chat E2E: Sender Keys protocol — một key per sender, distribute qua pairwise E2E channels
- Key Backup: Optional encrypted backup lên cloud (mã hóa bằng user's passphrase, PBKDF2)

Luồng tạo nhóm chat (Group Creation)

- User A gửi POST `/api/v1/conversations` `{type: 'group', name: 'Tên nhóm', member_ids: [B,C,D], avatar_url?}`

- Messaging Service validate: member_ids tồn tại, User A có kết nối với từng member, số thành viên ≤ 500
- Tạo document conversations trong MongoDB: {type:'group', participants:[{user_id:A, role:'admin'}, {user_id:B, role:'member'}, ...]}
- Publish event 'group.created' → Notification Service gửi thông báo 'A đã thêm bạn vào nhóm X' cho B, C, D
- Messaging Service emit WebSocket event 'conversation:new' đến tất cả participants online → client hiển thị nhóm mới trong danh sách chat
- Gửi system message đầu tiên: 'A đã tạo nhóm X' (type: 'system') vào MongoDB messages

Quản lý nhóm chat (Group Management)

Hành động	Ai có quyền	API	Hành vi hệ thống
Thêm thành viên	Admin / Creator	POST /conversations/:id/members	Validate kết nối, thêm vào participants, system message + notification
Xóa thành viên	Admin / Creator	DELETE /conversations/:id/members/:userId	Remove khỏi participants, thu hồi room, gửi system message
Rời nhóm	Bất kỳ member	DELETE /conversations/:id/members/me	Nếu admin duy nhất → tự động chuyển quyền cho member lâu nhất
Đổi tên nhóm	Admin	PATCH /conversations/:id {name}	Cập nhật MongoDB, system message, invalidate Redis cache
Đổi ảnh nhóm	Admin	PATCH /conversations/:id {avatar_url}	Upload S3, cập nhật MongoDB, broadcast WebSocket event
Phong Admin	Creator	PATCH /conversations/:id/members/:userId	Cập nhật role:'admin' trong participants array
Giải tán nhóm	Creator	DELETE /conversations/:id	Soft delete, system message, notification toàn bộ members

Ghim tin nhắn	Admin	POST /conversations/:id/pin	Lưu pinned_message_id trong conversations document
---------------	-------	-----------------------------	--

Phân quyền thành viên nhóm

Role	Gửi tin	Thêm/Xóa member	Đổi tên/Ảnh	Ghim tin nhắn	Phong Admin	Giải tán nhóm
Creator (người tạo)	✓	✓	✓	✓	✓	✓
Admin	✓	✓	✓	✓	X	X
Member	✓	X	X	X	X	X

6.3 Hệ thống kết nối nội bộ (Internal Connection System)

Hệ thống kết nối hỗ trợ ba loại quan hệ: Friend (hai chiều, cần accept), Follow (một chiều), và Colleague (tự động dựa trên Organization). Danh bạ tổ chức (Organization Directory) được sync từ HR system hoặc quản lý thủ công.

Luồng gửi kết bạn

1. User A gửi POST /api/v1/connections/request {addressee_id, type: 'friend'}
2. Validate: chưa có connection pending/accepted, addressee chưa block requester
3. Tạo record user_connections (status = 'pending')
4. Notification Service gửi push + in-app notification cho User B
5. User B accept → update status = 'accepted', tạo reverse record (B → A)
6. Publish event 'connection.accepted' → Newsfeed Service bắt đầu fan-out posts giữa hai user

Tìm kiếm & Gợi ý kết nối

- Search: Elasticsearch fuzzy search trên display_name, username, department — hỗ trợ tiếng Việt Unicode
- People You May Know: Graph-based suggestion — friends of friends (2nd degree connections)
- Organization-based: Cùng department, cùng team, cùng location — tự động gợi ý
- Mutual connections badge: Hiển thị số lượng bạn chung khi xem profile
- Rate limit: Max 50 friend requests / ngày để chống spam

6.4 Hệ thống thông báo (Notification System)

Kênh	Công nghệ	Use case
In-App (real-time)	WebSocket / Socket.IO	Tin nhắn mới, like, comment — user đang online
Push Notification	Firebase FCM (Android) + APNs (iOS)	Tin nhắn mới, friend request — user offline
Email	SendGrid / AWS SES	Email verification, password reset, weekly digest
SMS (OTP)	Twilio / Vonage	MFA verification, phone number confirmation

- Notification Preferences: User cấu hình on/off cho từng loại notification và từng kênh
- Batching: Gộp nhiều like trên cùng bài thành "A, B và 5 người khác đã thích bài viết của bạn"
- Quiet Hours: Không gửi push trong giờ ngủ (configurable per user, timezone-aware)
- Unread Counter: Badge count real-time trên app icon và in-app bell icon

7. Quy trình phát triển & CI/CD

7.1 Git Branching Strategy

Áp dụng GitFlow simplified: main (production), develop (staging), feature/*, hotfix/*. Mỗi feature branch tạo từ develop, merge qua Pull Request với required reviews.

Branch	Mục đích	Deploy tới	Protection Rules
main	Production code	Production (auto)	No direct push, require 2 approvals
develop	Integration branch	Staging (auto)	Require 1 approval, CI pass
feature/*	Feature development	Preview (PR)	Linked issue required
hotfix/*	Emergency fix	Production (fast-track)	Require 1 approval + CTO tag
release/*	Release preparation	Staging	Version bump, changelog required

7.2 CI Pipeline (GitHub Actions)

7. Trigger: Push to any branch hoặc Pull Request event
8. Lint & Format: ESLint + Prettier — reject nếu có lỗi
9. Type Check: TypeScript strict mode compilation
10. Unit Tests: Jest — coverage threshold $\geq 80\%$
11. Integration Tests: Supertest + Testcontainers (MySQL, Redis, MongoDB in Docker)
12. Security Scan: Snyk dependency check + Trivy container scan
13. Build: Docker image build (multi-stage, distroless base image)
14. Push: Tag và push image lên Container Registry (ECR / GCR)

7.3 CD Pipeline (ArgoCD — GitOps)

15. CI pipeline update Helm chart image tag trong GitOps repo
16. ArgoCD detect thay đổi → sync Kubernetes manifests
17. Rolling Update: Kubernetes triển khai pods mới, health check, drain pods cũ
18. Canary Release (cho major changes): 10% traffic → monitor 30 phút → 50% → 100%

19. Auto Rollback: Nếu error rate > 1% hoặc p95 latency > 2s trong 5 phút → rollback tự động

20. Slack notification: Deploy success/failure → #deploys channel

7.4 Testing Strategy

Loại Test	Tool	Coverage Target	Chạy khi
Unit Tests	Jest + @nestjs/testing	≥ 80% lines	Mọi push
Integration Tests	Supertest + Testcontainers	Critical paths	Mọi PR
E2E Tests	Playwright (Web) + Detox (Mobile)	Happy paths + edge cases	Trước release
Load Tests	k6 / Artillery	1000 concurrent users	Weekly + trước release
Security Tests	OWASP ZAP + Snyk	No critical/high vulns	Daily scan
Contract Tests	Pact	100% API contracts	Mọi PR thay đổi API

7.5 Monitoring & Observability

- Metrics (Prometheus): API latency (p50/p95/p99), error rate, request rate, active WebSocket connections, DB query time
- Logs (ELK Stack): Structured JSON logs → Filebeat → Logstash → Elasticsearch → Kibana dashboards
- Tracing (Jaeger): Distributed tracing across microservices — trace từ API Gateway → Auth → Post Service → DB
- Alerting (Grafana + PagerDuty): Error rate > 1% → Slack alert → 5% → PagerDuty page on-call engineer
- Uptime Monitoring (Pingdom / UptimeRobot): External health check mỗi 30 giây
- Real User Monitoring: Sentry (frontend error tracking) + web-vitals (Core Web Vitals)

8. Kế hoạch mở rộng & Tính năng tương lai

8.1 Kiến trúc mở rộng (Scalability)

- Horizontal Scaling: Mỗi microservice auto-scale độc lập dựa trên CPU/memory/request rate (Kubernetes HPA)
- Database Scaling: MySQL read replicas (MySQL Replication / AWS Aurora Read Replicas) cho read-heavy queries (newsfeed, profile), MongoDB sharding by conversation_id
- WebSocket Scaling: Socket.IO + Redis Adapter — thêm instance không cần sticky sessions
- Caching Layers: Redis Cluster (multi-node) cho session, feed cache, rate limiting — eviction policy: LRU

- CDN: Static assets + media cached tại edge locations — giảm latency cho users ở nhiều region
- Queue Scaling: Kafka partitions tăng theo throughput — consumer groups cho parallel processing
- Database Connection Pooling: ProxySQL hoặc MySQL Connection Pooling (built-in) — giới hạn connections tới MySQL

8.2 Các module tính năng tương lai

Kiến trúc microservices cho phép thêm module mới mà không ảnh hưởng core system. Mỗi module dưới đây là một service riêng, giao tiếp qua API Gateway và Message Broker:

Module	Mô tả	Tech bổ sung	Ưu tiên
Payment Service	Ví điện tử, chuyển tiền P2P, thanh toán in-app	Stripe / VNPay SDK, PCI DSS compliance	Giai đoạn 2
Marketplace	Mua bán sản phẩm/dịch vụ giữa users	Algolia search, escrow payment flow	Giai đoạn 3
Video/Voice Call	Gọi 1-1 và group call (audio + video)	WebRTC + Mediasoup SFU / Twilio SDK	Giai đoạn 2
Stories / Reels	Nội dung tạm thời (24h) và video ngắn	FFmpeg transcoding, Redis TTL	Giai đoạn 2
AI Features	Smart reply, content moderation, translation	OpenAI API / self-hosted LLM, TensorFlow	Giai đoạn 3
Analytics Dashboard	Thống kê engagement, user growth, content performance	ClickHouse + Metabase / Redash	Giai đoạn 2
Admin Panel	Quản lý users, content moderation, system config	React Admin / Retool	Giai đoạn 1.5
Bot Platform	Chatbot API cho developers tích hợp (giống Slack bots)	Webhook + OAuth app platform	Giai đoạn 3

8.3 Chuẩn bị cho tích hợp thanh toán

Mặc dù Payment Service thuộc giai đoạn 2, kiến trúc giai đoạn 1 đã đặt nền tảng để tích hợp nhanh:

- User Service đã có KYC-ready fields (phone_verified, email_verified) — chỉ cần thêm identity verification
- API Gateway đã có rate limiting và authentication — Payment endpoints chỉ cần thêm middleware layer
- Event-driven architecture sẵn sàng: payment.completed event → Notification, Transaction Log, Analytics
- Database-per-service: Payment Service sẽ có database riêng (MySQL) cho wallets, transactions — không couple với User DB
- Encryption infrastructure (Vault, TLS, AES-256) đã sẵn sàng cho PCI DSS data-at-rest requirements

- Audit logging framework đã có — mở rộng cho financial audit trail

8.4 Roadmap phát triển

Giai đoạn	Thời gian	Deliverables
Phase 1 — MVP	Tháng 1–4	Auth, Profile, Post/Feed, Messaging (1-1 + group), Connections, Notifications, Basic Admin
Phase 1.5 — Hardening	Tháng 5–6	E2E encryption, load testing, security audit, Admin Panel v1, bug fixes
Phase 2 — Growth	Tháng 7–10	Payment/Wallet, Video Call, Stories, Analytics Dashboard, push notification optimization
Phase 3 — Platform	Tháng 11–14	Marketplace, AI features, Bot Platform, public API, third-party integrations
Phase 4 — Scale	Tháng 15+	Multi-region deployment, advanced recommendation engine, enterprise features

8.5 Đề xuất cơ cấu đội ngũ (Phase 1)

Vai trò	Số lượng	Trách nhiệm chính
Tech Lead / Architect	1	Kiến trúc hệ thống, code review, technical decisions
Backend Engineers	2–3	Microservices development (NestJS), database, API
Frontend Engineers	2	Web (Next.js) + Mobile (React Native)
DevOps / SRE	1	CI/CD, Kubernetes, monitoring, infrastructure
UI/UX Designer	1	Wireframes, design system, user research

QA Engineer	1	Test automation, manual testing, security testing
Product Manager	1	Roadmap, prioritization, stakeholder management

8.6 Chiến lược tạo doanh thu (Monetization Strategy)

Nền tảng được thiết kế theo mô hình “Open Source Core + Commercial Enterprise” — mã nguồn mở hoàn toàn cho cộng đồng, người dùng cá nhân luôn miễn phí (có giới hạn) trên bản hosted để cạnh tranh với Zalo, Telegram. Doanh thu đến từ doanh nghiệp muốn tự kiểm soát dữ liệu (self-hosted), giao dịch, và hệ sinh thái nền tảng theo từng giai đoạn tăng trưởng:

Giai đoạn 1 — Tăng trưởng user base (6 tháng đầu): Toàn bộ tính năng miễn phí. Mục tiêu duy nhất là đạt ngưỡng MAU đủ lớn để các kênh doanh thu phát huy hiệu quả.

Giai đoạn 2 — Mô hình Open Source Enterprise (từ Phase 2, tháng 7–10): Doanh nghiệp mua license thương mại hoặc gói dịch vụ để tự triển khai (self-hosted), kết nối database nội bộ, kiểm soát toàn bộ dữ liệu trên hạ tầng riêng. Cá nhân vẫn miễn phí (limited tier) trên bản hosted. Các gói doanh nghiệp bao gồm: (1) Gói Team — thu phí theo deployment/năm, workspace riêng, quản lý thành viên tập trung, SSO/LDAP integration, các module thêm (Meeting, Calendar, Notes, Tasks); (2) Gói Enterprise — on-premise hoặc private cloud deployment, SLA cam kết uptime, custom branding, hỗ trợ kỹ thuật ưu tiên, toàn bộ module đặc quyền. Đây là nguồn doanh thu ổn định nhất vì doanh nghiệp kiểm soát dữ liệu và khó rời bỏ sau khi đã tích hợp vào quy trình.

Bảng so sánh tính năng theo gói dịch vụ:

Tính năng / Giới hạn	Free (Cá nhân)	Enterprise
Nhắn tin 1-1	✓ Không giới hạn	✓ Không giới hạn
Kích thước nhóm chat	Tối đa 50 người	Không giới hạn
Lưu lịch sử tin nhắn	1 năm	Vĩnh viễn + export log
Workspace tổ chức	Không có	Multi-workspace
Không có	Không có	Admin console đầy đủ, audit log
Xác thực & Bảo mật	Email / Social Login	LDAP / SAML, 2FA bắt buộc, IP whitelist
Storage file & media	2 GB / người dùng	Không giới hạn / tự host
API & Tích hợp	Không có	Full API, Bot Platform, Zapier/n8n
Custom branding	Không có	Toàn bộ giao diện (white-label)
On-premise / Private cloud	Không có	✓ Hỗ trợ

SLA uptime	Best-effort	99.9%
Hỗ trợ kỹ thuật	Community support	Dedicated support ưu tiên
Giá (tham khảo)	Miễn phí	Thỏa thuận theo hợp đồng

Giai đoạn 3 — Marketplace & Transaction Fee (từ Phase 3, tháng 11–14): Thu phí giao dịch 2–5% trên mỗi đơn hàng qua Marketplace và phí chuyển tiền / rút tiền từ Payment Wallet.

Giai đoạn 4 — API Platform & Đối tác (từ Phase 4, tháng 15+): Mở Bot Platform API cho developer thứ ba (gói subscription hoặc revenue share). Xem xét quảng cáo native chỉ khi nền tảng đạt quy mô đủ lớn và không làm xấu trải nghiệm người dùng.